

HPC Architectures: Accelerated Sharpen and STREAMS

David Henty and Adrian Jackson

1 Introduction

The purpose of this practical session is to investigate running programs on the GPU nodes of ARCHER2. You will be given a GPU-enabled version of the parallel CPU sharpen program that you ran at the start of the course. You can now compare the performance of the new GPU code to the old CPU version. If you want to find out the details of the hardware on ARCHER2 (AMD CPUs and AMD GPUs) then see <https://docs.archer2.ac.uk/user-guide/gpu/>.

We will also provide a GPU version of the STREAMS memory benchmark (<https://www.cs.virginia.edu/stream/>). This has been implemented using two different methods for offloading work on to a GPU, and can be used to explore the memory performance of the GPUs on ARCHER2.

2 Sharpen

2.1 Compiling and Running

First clone the repository:

```
git clone https://github.com/davidhenty/sharpen.git
```

(if you have already cloned sharpen you should be able to update using `git pull` within the repository).

2.2 CPU Performance

We will first run the CPU version parallelised using OpenMP. To make a fair comparison we will run on the host CPUs on the GPU nodes as these are more modern than the CPUs in the standard compute nodes. Note that these newer CPUs have 32 CPU-cores rather than 128.

Compile and run as follows:

```
cd sharpen/src/C-OMP
sbatch sharpengpuhost.job
```

The batch script is set up to run in a reserved GPU queue that is active for the practical session from 9-10 Friday 7th November. **Outside of that time (or if you are a student from an external programme, not the MSc in HPC or IVHPC) you will need to edit the batch script appropriately.**

The script is set up to run the sharpen code on a range of thread counts from 1 to 32. How well does it scale? How much faster is it than using 32 threads on the standard compute nodes (using `sharpen.slurm`). We are only interested in the calculation time which excludes initialisation, IO etc. which are not parallelised.

2.3 GPU version

The GPU version is parallelised using HIP, a low-level programming model specific to AMD GPUs (but very like Nvidia's CUDA). The user writes a small kernel function to be executed by a single GPU thread. The program runs on the host CPU but there is special syntax to launch the kernel in parallel across large numbers of GPU threads (GPUs have thousands of simple cores compared to CPUs which have tens of general-purpose cores). Here, the kernel is the filter function that is applied to every pixel in the fuzzy image. Since the CPU and GPU have separate memory spaces there are also routines to allocate GPU memory and transfer data from/to the CPU. HIP does not have good support for multidimensional arrays so the HIP filter kernel `dosharpenpixel.c` looks horrible compared to the equivalent CPU version!

2.3.1 Default setup

To compile and run the program on a single GPU:

```
cd ../C-HIP
sbatch sharpengpu.job
```

You should always check that the output image `sharpened.pgm` is identical to the CPU version: runtime errors on the GPU are not always trapped properly so a program can run to completion but produce incorrect output. This will give unreliable performance results.

Here we are only interested in the calculation time.

How much faster is the GPU version than a single CPU-core? How much faster is it than using all 32 CPU-cores? Do you think this additional performance is worth the cost (note that you may find it hard to find reliable price information on CPUs and GPUs for HPC!).

The performance increase we get for `sharpen` is unusually high as it is almost entirely CPU-bound rather than memory-bound: it computes a very large number of exponential functions. Although GPUs also have much better memory bandwidth than CPUs, the performance increase for a real program is unlikely to be as large as you observe here.

2.3.2 Varying the number of threads

The code that launches the kernel is in `dosharpen.cpp` - the relevant parts are:

```
dim3 nthread = {16, 16, 1}; // 256 in a 16x16 grid
...
dosharpenpixel<<<nblock, nthread>>>(nx, ny, d, d_conv, d_fuzzyp);
```

The `<<nblock, nthread>>` syntax launches the kernel function `dosharpenpixel()` multiple times, each running on a block of `nthread` GPU threads. Each thread operates on a single pixel: the number of blocks is automatically chosen to cover the whole image. For a 2D image it is convenient to think of each thread block as a 2D grid: here we have 256 threads in each block arranged in a 16x16 grid.

GPUs use a Single Instruction Multiple Thread (SIMT) architecture with threads arranged in “warps” where all threads in a warp must run the same instruction. If the number of threads in a block is smaller than the warp size then some of the threads in each warp will be idle leading to reduced performance.

Vary the number of threads in a block by editing `dim3 nthread = 16,16,1` to different values `{1,1,1}`, `{2,2,1}`, ..., `{32,32,1}`. Each time you must recompile and resubmit: wait until each job completes before recompilation, otherwise you may not be running the program you expect!

From how the performance varies as a function of the the number of threads in a block, how many threads do you think there are in each warp?

3 STREAMS

The STREAMS benchmark tries to probe the memory bandwidth of a system it is run on. Often used to investigate the memory bandwidth of main memory or CPU caches, it can also be adapted to evaluate GPU memory performance. On CPUs what memory it actually benchmarks (i.e. caches or main memory) is controlled using the size of the data set being benchmarked. If the data set is small enough to fit into one of the levels of cache, then you can use STREAMS to map cache bandwidth. If the data set is significantly larger than the biggest cache size, i.e. 3-4 times bigger than cache, then it will give performance estimates for main memory bandwidth.

We have put together a GPU version of the benchmark: you can clone the repository using:

```
git clone https://github.com/davidhenty/archer2-stream.git
```

3.1 HIP Performance

The first implementation is done using HIP. You can compile and run this as follows:

```
cd archer2-stream/src
make -f HIP.make
sbatch streamgpu.job
```

This should give you the performance from the GPU memory using HIP.

What bandwidth do you get using the GPU? How does this compare to the previous results for the CPU version on ARCHER2 (where we saw about 220 GB/s for Triad)?

The current code is configured to use around 640 MB per array or around 1.9 GB in total (the same size we used previously for the CPU version). You can modify this by changing this line in `main.cpp`:

```
unsigned int ARRAY_SIZE = 80000000;
```

If you make this larger you may also want to modify the number of times the benchmark is repeated, i.e. this line in `main.cpp`:

```
unsigned int num_times = 100;
```

Try changing the size of the memory required, recompiling and re-running the benchmark. Do you see any performance variation with different memory sizes?

3.2 Other implementations

We have also provided some other implementations of the benchmark, in particular one using OpenMP for the offloading functionality. These can be compiled and run alongside the HIP benchmark to see if the different methods have varying performance.

For the OpenMP version you can build it like this:

```
module load rocm
module load craype-accel-amd-gfx90a
module load craype-x86-milan
make -f OpenMP.make
```

The `sharpgpu.job` script has code to run the OpenMP version that is currently commented out.

Compile and run the OpenMP version: are there any performance differences compared to the lower-level HIP version?